

Nama: Nashira Salima Firmansyah
NIM: J0403251056

Hasil Program

16. Praktikum 1 - Membuat Adjacency Matrix

Adjacency Matrix:

```
    0 1 2 3
0 : 0 1 1 0
1 : 1 0 1 0
2 : 1 1 0 1
3 : 0 0 1 0
```

Arti setiap baris:

Baris 0: node 0 terhubung dengan node 1 dan 2.
Baris 1: node 1 terhubung dengan node 0 dan 2.
Baris 2: node 2 terhubung dengan node 0, 1, dan 3.
Baris 3: node 3 terhubung dengan node 2.

17. Praktikum 2 - Membuat Adjacency List

Adjacency List:

```
A -> ['B', 'C']
B -> ['A', 'D']
C -> ['A', 'D']
D -> ['B', 'C']
```

18. Praktikum 3 - Konversi Matrix ke List

Matrix awal:

```
    A B C D
A : 0 1 1 0
B : 1 0 1 0
C : 1 1 0 1
D : 0 0 1 0
```

Hasil konversi ke adjacency list:

```
A -> ['B', 'C']
B -> ['A', 'C']
C -> ['A', 'B', 'D']
D -> ['C']
```

19. Praktikum 4 - Studi Kasus Dunia Nyata

Digit akhir NIM: 6

Studi kasus : Peta Kota

Node / Vertex:

- Bogor
- Depok
- Jakarta
- Bandung
- Sukabumi

Edge / Hubungan antar kota:

- Bogor terhubung dengan Depok
- Bogor terhubung dengan Sukabumi
- Depok terhubung dengan Jakarta
- Depok terhubung dengan Sukabumi
- Jakarta terhubung dengan Bandung
- Bandung terhubung dengan Sukabumi

Adjacency List Peta Kota:

```
Bogor -> ['Depok', 'Sukabumi']
Depok -> ['Bogor', 'Jakarta', 'Sukabumi']
Jakarta -> ['Depok', 'Bandung']
Bandung -> ['Jakarta', 'Sukabumi']
Sukabumi -> ['Bogor', 'Depok', 'Bandung']
```

Adjacency Matrix Peta Kota:

	Bogor	Depok	Jakarta	Bandung	Sukabumi
Bogor	0	1	0	0	1
Depok	1	0	1	0	1
Jakarta	0	1	0	1	0
Bandung	0	0	1	0	1
Sukabumi	1	1	0	1	0

16. Membuat Adjacency Matrix

Adjacency matrix adalah representasi graph dalam bentuk matriks dua dimensi berukuran $N \times N$, di mana N adalah jumlah node. Nilai 1 menunjukkan adanya hubungan antar node, sedangkan 0 menunjukkan tidak adanya hubungan.

Pada kode, fungsi `buat_matrix_tidak_berarah()` digunakan untuk membuat matrix graph tidak berarah. Setiap edge akan mengisi dua posisi sekaligus (simetris), yaitu (i, j) dan (j, i) .

17. Membuat Adjacency List

Adjacency list adalah representasi graph dalam bentuk struktur data dictionary, di mana setiap node memiliki daftar node lain yang terhubung langsung dengannya.

Pada kode, fungsi `buat_list_tidak_berarah()` digunakan untuk membangun adjacency list. Setiap edge akan ditambahkan ke kedua node, karena graph bersifat tidak berarah.

18. Konversi Adjacency Matrix ke Adjacency List

Konversi dilakukan untuk mengubah representasi matrix menjadi list. Proses ini dilakukan dengan membaca setiap baris pada matrix dan mencatat node yang memiliki nilai 1 sebagai tetangga.

Fungsi `matrix_ke_list()` melakukan iterasi pada matrix dan menghasilkan adjacency list yang setara.

19. Studi Kasus Dunia Nyata - Peta Kota



Penjelasan Node dan Edge

Pada studi kasus ini, graph merepresentasikan hubungan antar kota.

- **Node (vertex):**
Bogor, Depok, Jakarta, Bandung, dan Sukabumi
- **Edge (hubungan):**
 - Bogor terhubung dengan Depok
 - Bogor terhubung dengan Sukabumi
 - Depok terhubung dengan Jakarta
 - Depok terhubung dengan Sukabumi
 - Jakarta terhubung dengan Bandung
 - Bandung terhubung dengan Sukabumi

Graph ini bersifat **tidak berarah**, sehingga setiap hubungan berlaku dua arah.

Analisis Adjacency List vs Adjacency Matrix

1. **Adjacency List**
 - Lebih hemat memori karena hanya menyimpan hubungan yang ada
 - Cocok untuk graph dengan jumlah edge sedikit (*sparse graph*)
 - Efisien dalam menelusuri tetangga suatu node
2. **Adjacency Matrix**
 - Memudahkan pengecekan hubungan antar node secara langsung

- Akses data lebih cepat (kompleksitas $O(1)$)
- Membutuhkan memori lebih besar karena ukuran tetap $N \times N$

Kesimpulan

Representasi graph dapat dilakukan menggunakan adjacency matrix maupun adjacency list. Pada studi kasus peta kota, adjacency list lebih efisien digunakan karena jumlah hubungan antar kota relatif sedikit sehingga lebih hemat memori dan fleksibel. Sementara itu, adjacency matrix lebih unggul dalam kecepatan akses, namun kurang optimal untuk graph berukuran besar dengan koneksi yang tidak terlalu banyak.